

2024年度後期第4回輪読会 【物体検出】

ConvNeXt ConvNeXt V2

京都大学理学部2回

千葉 一世

- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- ConvNeXt V2の概要
- FCMAE, GRN
- ConvNeXt V2の性能評価

- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- ConvNeXt V2の概要
- FCMAE, GRN
- ConvNeXt V2の性能評価

ConvNeXtの概要

2020年にViTが登場してから、画像分野にTransformerが台頭し、Swin Transformerではウィンドウをずらすという畳み込みの考えを導入して成功を収めた。

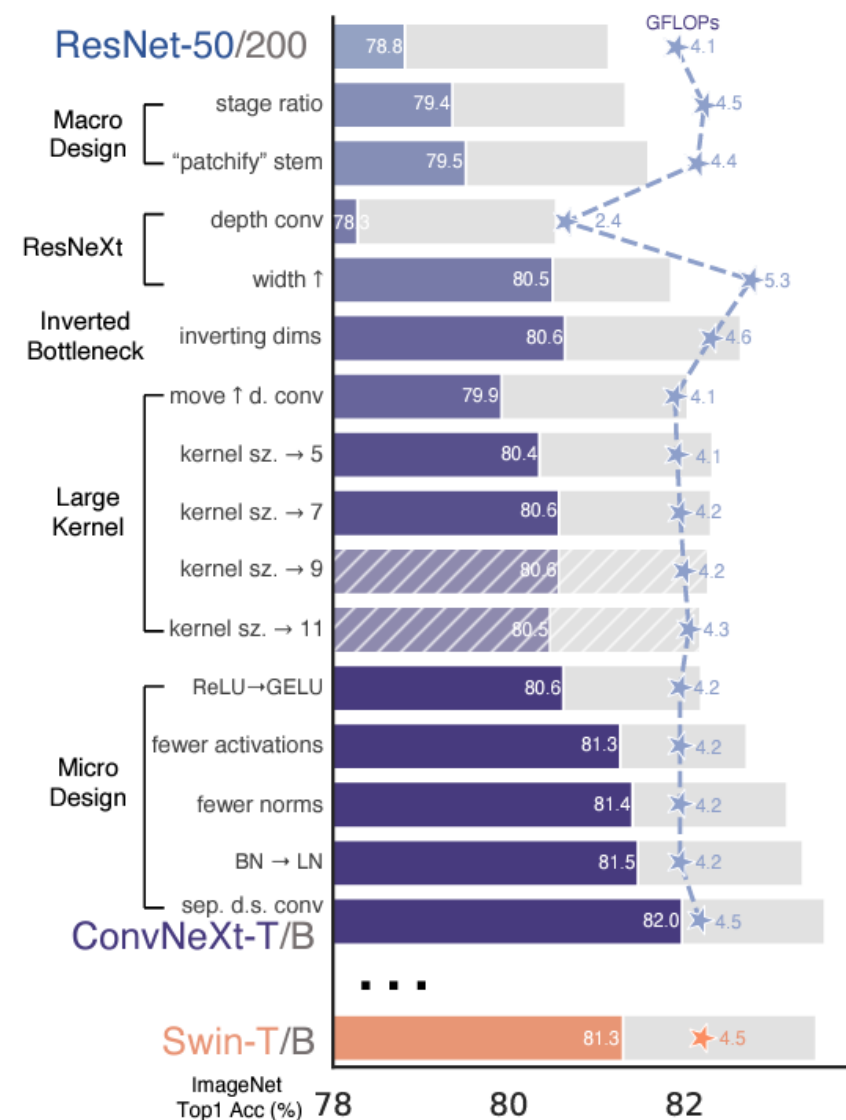
- ・ Transformer系の問題点は
解像度の二乗で増える計算量や、帰納バイアスが無いこと
- ・ 純粋なCNN系のモデルで、Transformerを超えるモデルの開発を目指す。
- ・ ResNetをベースとして、Swin Transformerなどの最新のモデルで用いられている手法を取り入れ、現代のCNN系モデルを構築していく
- ・ 驚くべきことに、Swin Transformerで用いられる手法のほとんどが有効に働いた

実験は、ImageNet-1Kのデータセットを用いて学習し、1位予測の精度によって評価した

ConvNeXtの概要

■ ResNetからの変更点

- 学習方法の見直し
- 様々なデータ拡張と正則化
- ステージ毎のブロック数の見直し
- パッチ化の導入
- Depthwise Convの導入とチャンネル数の増加
- 反転ボトルネックへ変更
- 7×7の大きなカーネルサイズを使用
- 活性化関数をReLUからGeLUにし、数を減らす
- バッチ正規化をレイヤー正規化にする



- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- ConvNeXt V2の概要
- FCMAE, GRN
- ConvNeXt V2の性能評価

ResNetからConvNeXtへ

■ 学習手法の変更

1. オプティマイザーにAdamWを用いる
2. 学習量を90エポックから300エポックに増やす
3. Mixup, Cutmix, RandAugment, Random Erasing のデータ拡張を行う
4. Stochastic Depthの導入
5. Label Smoothingの導入
6. Layer scaleの導入

精度は76.1% → 78.8%に上昇

(pre-)training config	ConvNeXt-T/S/B/L ImageNet-1K 224 ²	ConvNeXt-T/S/B/L/XL ImageNet-22K 224 ²
weight init	trunc. normal (0.2)	trunc. normal (0.2)
optimizer	AdamW	AdamW
base learning rate	4e-3	4e-3
weight decay	0.05	0.05
optimizer momentum	$\beta_1, \beta_2=0.9, 0.999$	$\beta_1, \beta_2=0.9, 0.999$
batch size	4096	4096
training epochs	300	90
learning rate schedule	cosine decay	cosine decay
warmup epochs	20	5
warmup schedule	linear	linear
layer-wise lr decay [6, 12]	None	None
randaugment [14]	(9, 0.5)	(9, 0.5)
mixup [90]	0.8	0.8
cutmix [89]	1.0	1.0
random erasing [91]	0.25	0.25
label smoothing [69]	0.1	0.1
stochastic depth [37]	0.1/0.4/0.5/0.5	0.0/0.0/0.1/0.1/0.2
layer scale [74]	1e-6	1e-6
head init scale [74]	None	None
gradient clip	None	None
exp. mov. avg. (EMA) [51]	0.9999	None

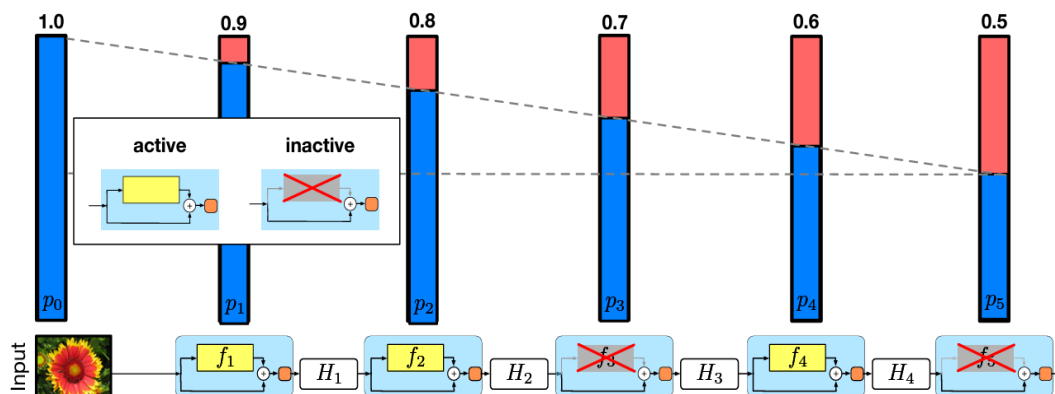
Stochastic Depth

確率的にブロックを丸ごとスキップする

- ・ 学習の速度が上がる
- ・ モデル全体の表現力向上

$$H_\ell = \text{ReLU}(b_\ell f_\ell(H_{\ell-1}) + \text{id}(H_{\ell-1})).$$

$$b_\ell \in \{0, 1\}$$

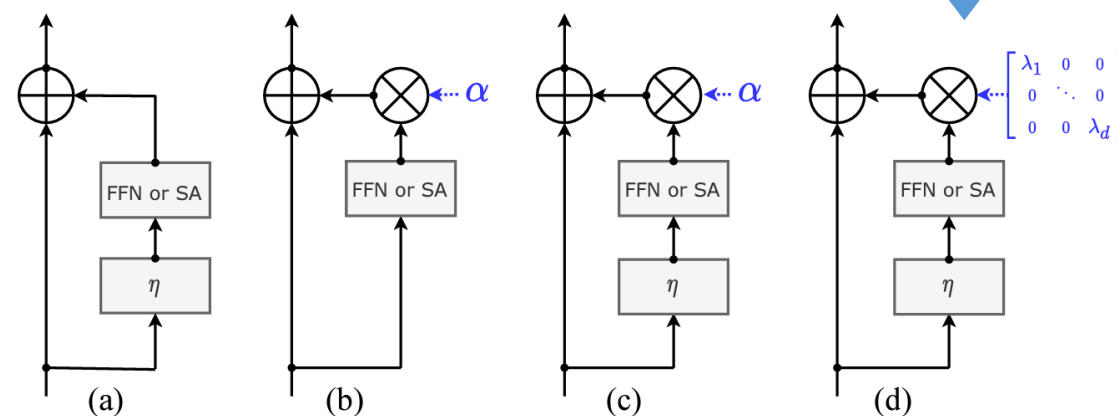


Layer Scale

残差ブロック内の出力のスケールを調整する

- ・ 層を深くしても学習が安定する
- ・ 初期値を小さくすることで、学習が徐々に進むようにする

各チャンネルに異なる値をかける



ResNetからConvNeXtへ

■ ステージ毎のブロック数の見直し

ResNetでは、経験的にブロック数を決めており、順に(3, 4, 6, 3)であったが、Swin Transformerでは、(2, 2, 6, 2) や (2, 2, 18, 2) のブロック数が用いられている。

3つ目のステージは、下流タスクへの応用に重要であるためにブロック数を多く割いているが、ResNetではそこまで多く設定されていない。



Swin Transformerに倣い、ブロック数として(3, 3, 9, 3)を採用する

精度は78.8% → 79.4%に向上

ResNetからConvNeXtへ

■ パッチ化

画像処理では、高解像度のデータを徐々にダウンサンプリングしていくが、特に、初めのStem層で大きくダウンサンプリングを行う

ResNetでは、 7×7 のストライド2の畳み込みとMaxPoolingでダウンサンプリングするが、Swin Transformerの、 4×4 重複無しでパッチに分割する手法を取り入れ、 4×4 でストライド4の重複無し畳み込みを用いる

これで、79.4% → 79.5%に

- 何故情報が減っているはずのパッチ化で精度が下がらないのか

画像には無駄な部分が多く、 4×4 程まで拡大すると、その16ピクセルはほとんど同じ情報しかもっていない。

つまり、情報量は誤差レベルでしか変化していない

ResNetからConvNeXtへ

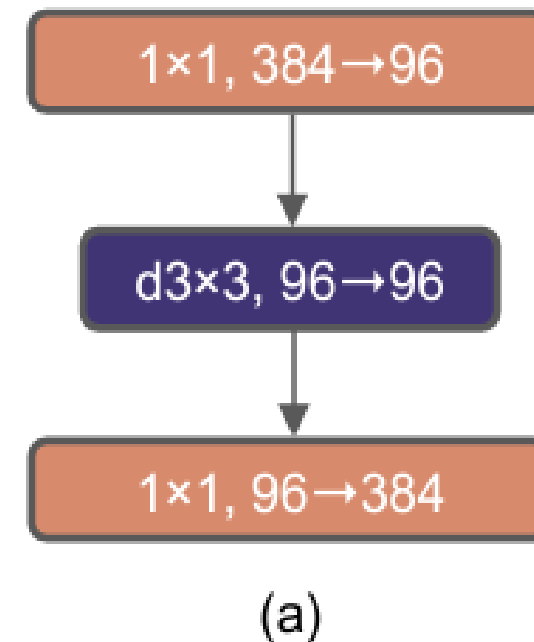
■ Depthwise・Pointwise 畳み込みに変更

ResNeXtのモデル設計から、グループ化畳み込みを取り入れ「グループ数とチャンネル数を多くする」という考えから、グループ数を最大限増やしたDepthwise畳み込みを採用する。Depthwise畳み込みに変更するだけでは精度が大きく落ちるので、その後に 1×1 畳み込みを合わせて用いる。

チャンネル数もSwin Transformerと同じ96まで増やす

Depthwise・Pointwise畳み込みは、純粋な畳み込みが本来一度に行っている空間方向とチャンネル方向の集約を分離して行っており、Transformerにおいて、Attentionが空間的な繋がりを計算した後に、MLPでチャンネル方向の計算をしている事と上手く対応した構造になっている

これにより79.5% → 80.5%に



ResNetからConvNeXtへ

■ 反転ボトルネック(Inverted Bottleneck)

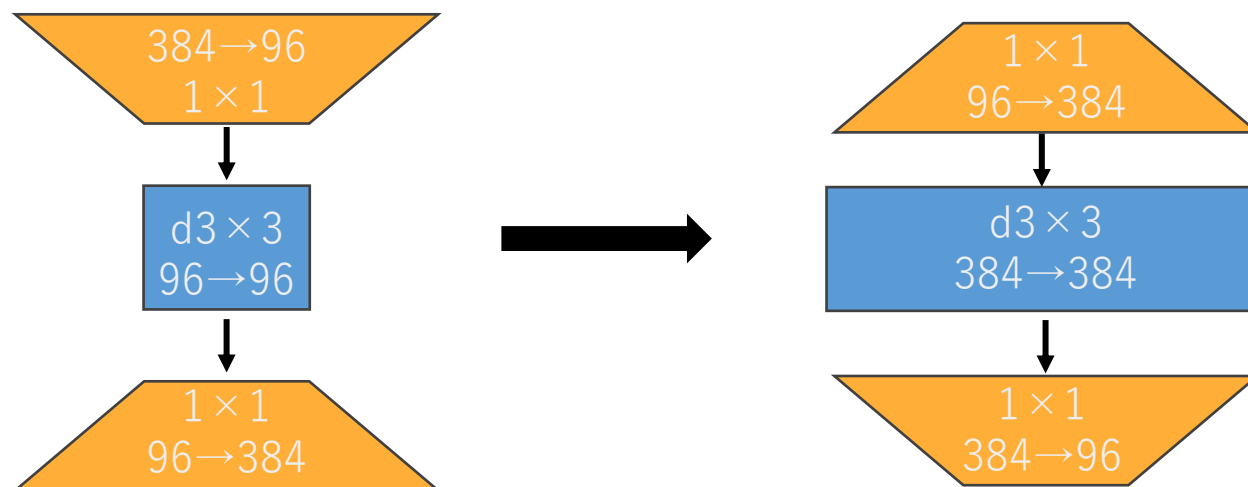
- Swin TransformerのMLPでは、中間層の次元が4倍になっている。



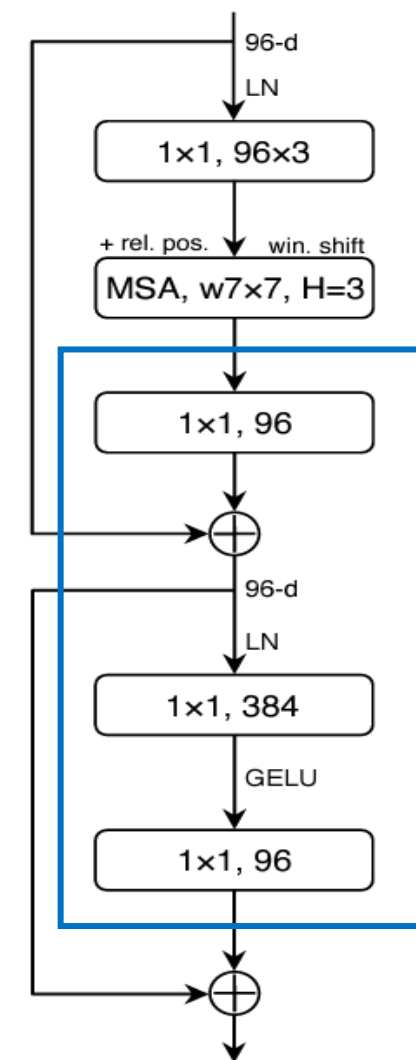
- 間のチャンネル数だけを増やす**反転ボトルネック構造**を用いる
- FLOPsが減少し、精度も80.5% → **80.6%**に上がった

特に、大きなモデルの ResNet-200/Swin-Bの領域では

さらに顕著に精度が上がり、81.9% → **82.6%**となった



Swin Transformer



ResNetからConvNeXtへ

■ カーネルサイズを大きくする

- ConvNetとVision Transformerの最大の違いは、畳み込みが局所的な情報を層を経る毎に積み重ねていくのに対して、Attentionは、各層毎に全体の特徴を集約できる



- カーネルサイズを大きくすることによって、より広い受容野の獲得を目指す
- カーネルサイズを大きくすると計算量が増えるので、ブロックの一層目のチャンネル数が少ない部分に入れる。
これは、TransformerがAttention → MLPの順になっている事と合致する
- 実験の結果、**7×7**が一番精度が良くなった
精度としては、**80.6%**のまま変化なし

model	IN-1K acc.	GFLOPs
kernel size → 5	80.35 ± 0.08	4.10
kernel size → 7	80.57 ± 0.14	4.15
kernel size → 9	80.57 ± 0.06	4.21
kernel size → 11	80.47 ± 0.11	4.29

model	IN-1K acc.	GFLOPs
kernel size → 5	82.32	14.70
kernel size → 7	82.30	14.81
kernel size → 9	82.27	14.95
kernel size → 11	82.18	15.13

ResNetからConvNeXtへ

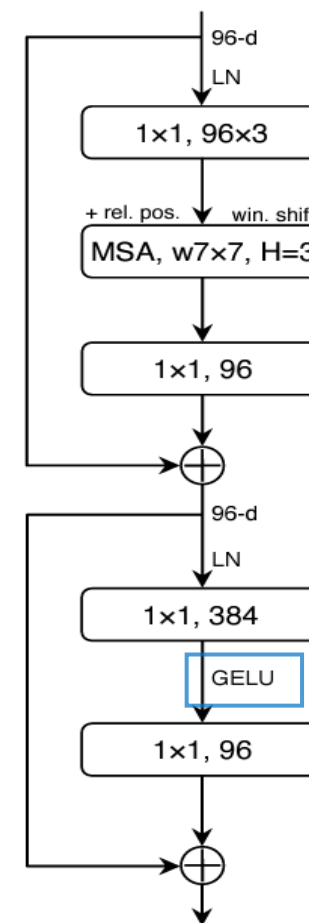
■ 活性化関数を変更

- Transformerでは、ブロック内に活性化関数がMLP層の一つしか含まれないが、ResNetでは畳み込み層毎に活性化関数があり、活性化関数の数が多い。

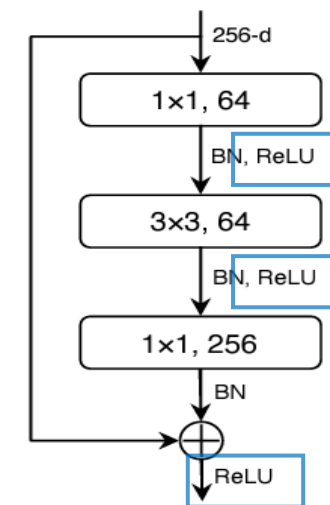


- 1ブロックに1つの活性化関数を使用し、ReLUをGeLUに変更する。
- その結果、80.6% → 81.3%となり、Swin Transformerに匹敵する精度となった。

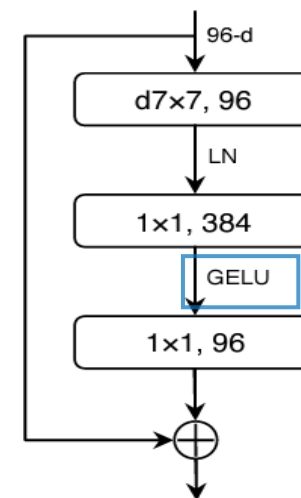
Swin Transformer Block



ResNet Block



ConvNeXt Block



ResNetからConvNeXtへ

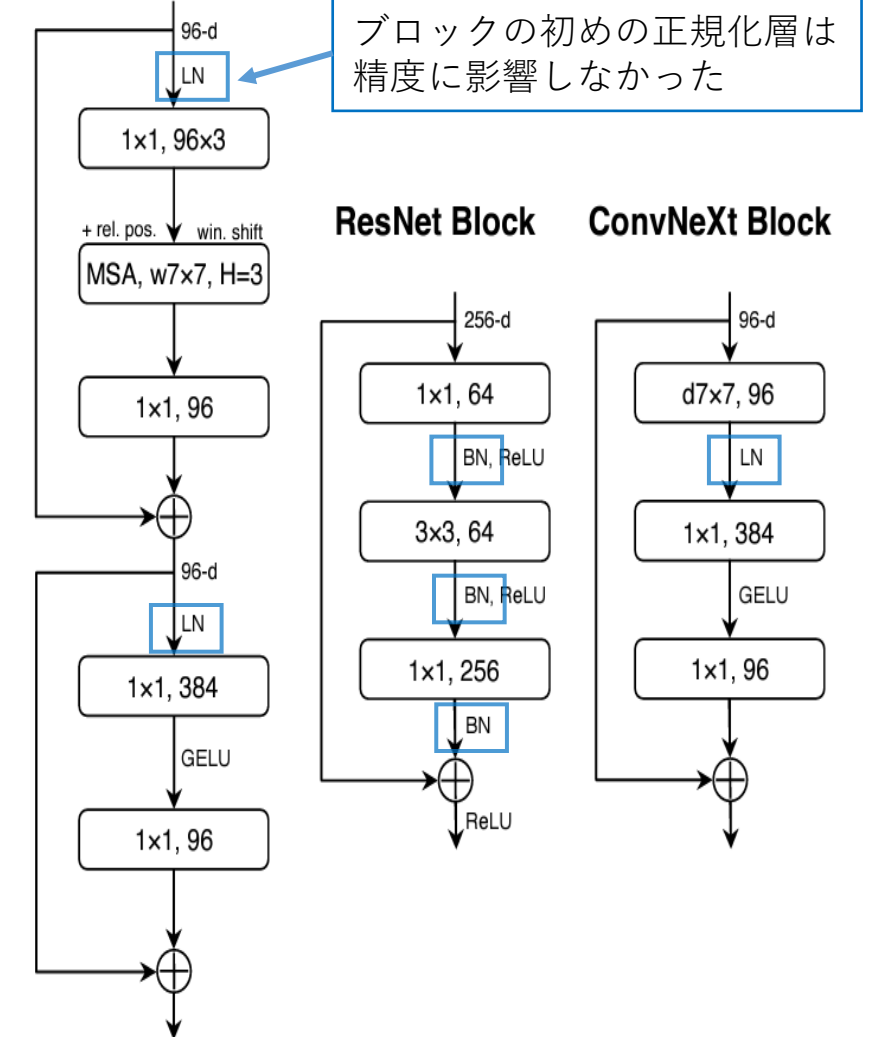
■ バッチ正規化を層正規化に変更し、1ブロックに1つとする

- 活性化関数と同様、正規化層についてもTransformerではResNetと比べて少なく、層正規化を用いている



- ・ **バッチ正規化層を一つに減らす**
- ・ さらに、**バッチ正規化を層正規化に置き換える**
- ・ ResNetでは、バッチ正規化を層正規化に置き換えても上手くいかないが、ViTに似たアーキテクチャに大きく変更したからか、層正規化によって精度がわずかに向上し、81.3% → **81.5%**に

Swin Transformer Block



ResNetからConvNeXtへ

■ ダウンサンプリング層を重複無し畳み込みに変更

各ステージの初めのダウンサンプリング層についても、

パッチ化のように重複のない 2×2 畳み込みを用いる

そのままでは学習が発散してしまうが、ダウンサンプリングの前に層正規化を入れることによって学習が安定する

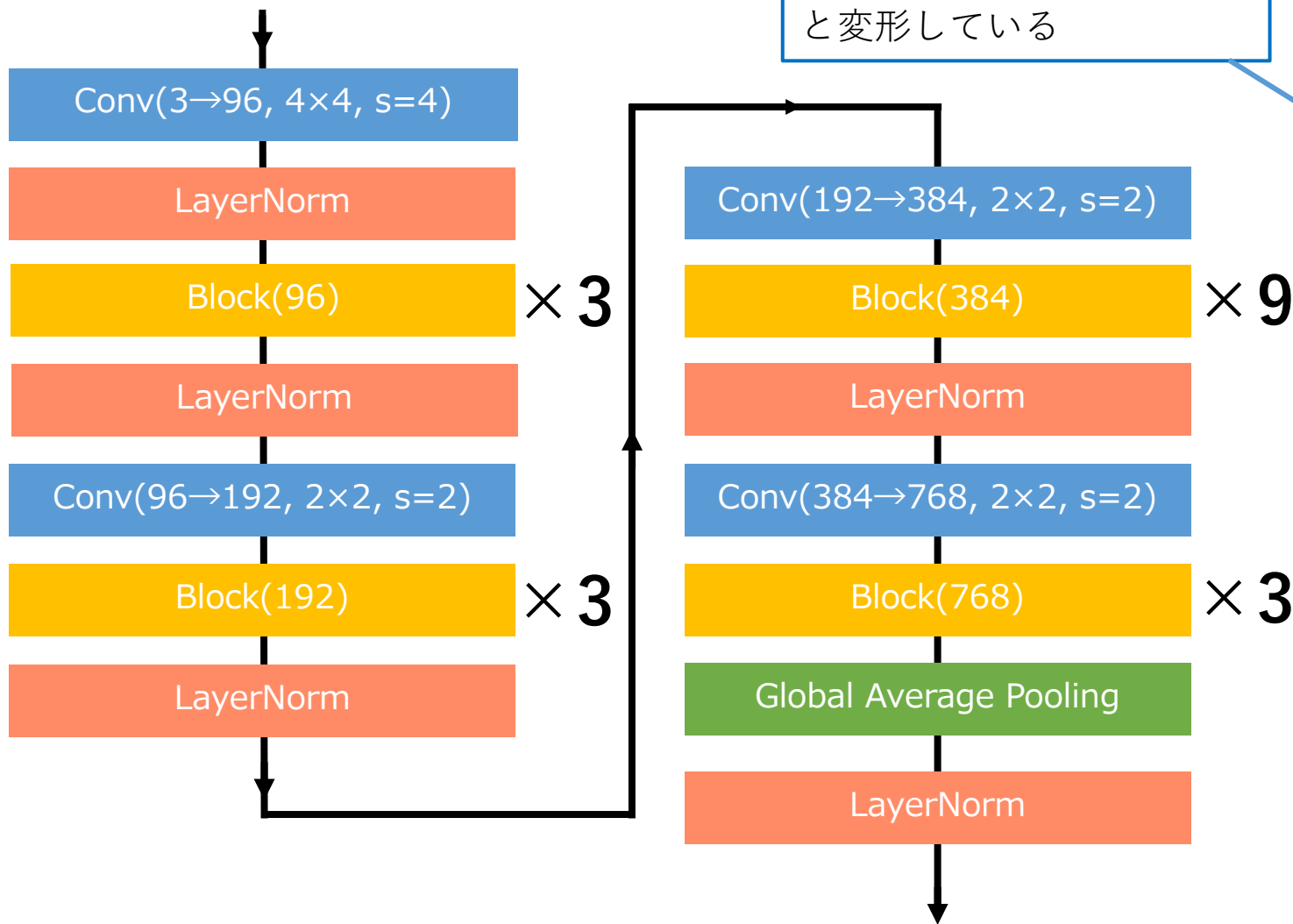
解像度が変化する際には、層正規化を入れると学習が安定することがわかったので、

さらに、stem層の後と最後のグローバル平均プーリングの後に1つずつ層正規化を入れる。

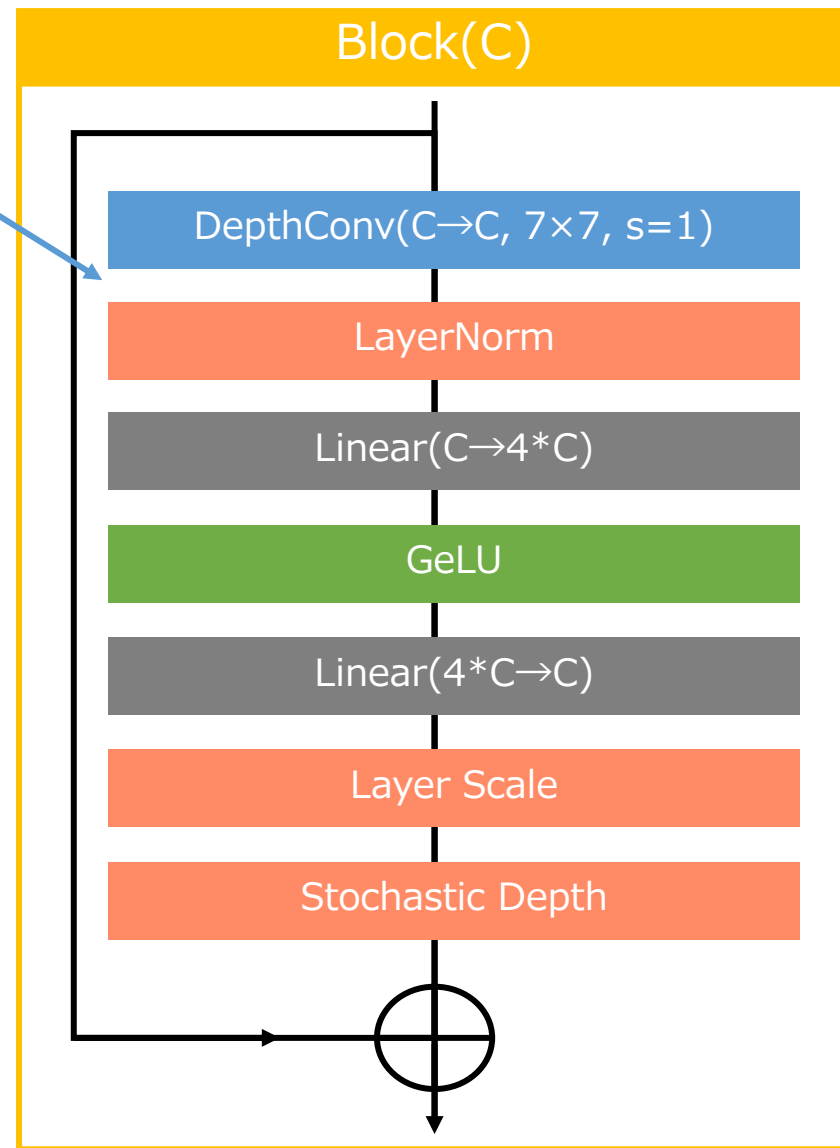
精度は81.5% → 82%まで上昇した

ResNetからConvNeXtへ

最終的なモデル構造(Tiny)



Pointwise畳み込みを
線形層で実装するために、
(B, C, H, W)→(B, H, W, C)
と変形している

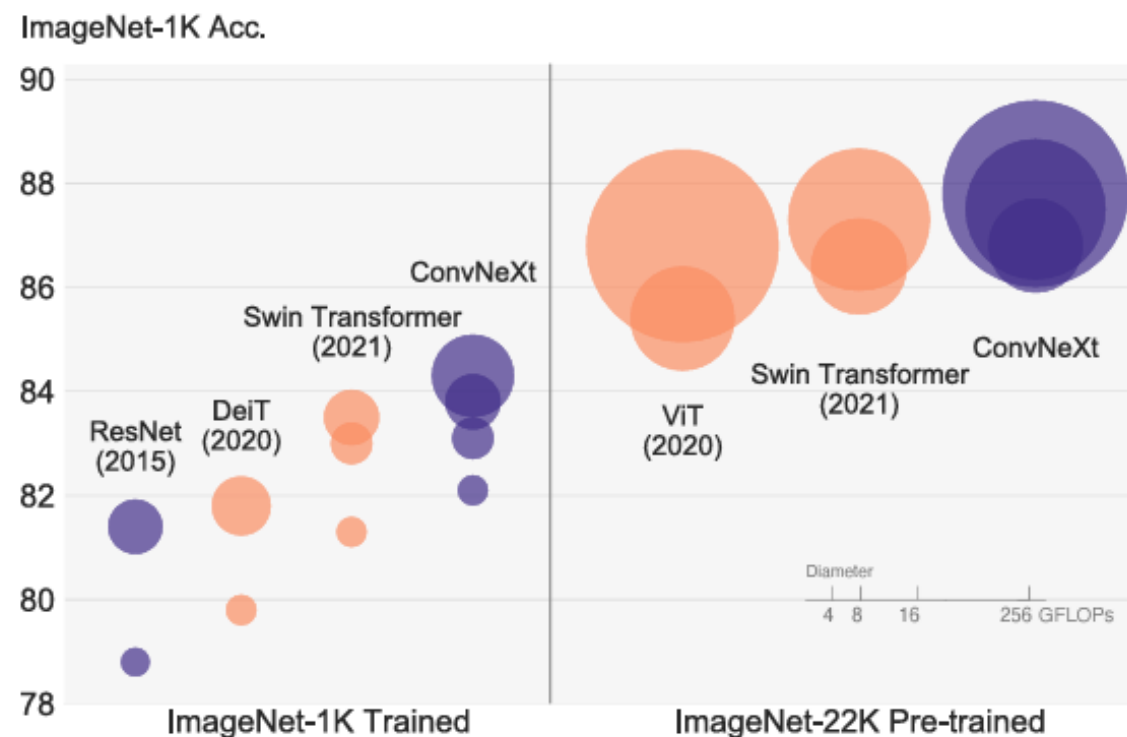


- ConvNeXtの概要
- ResNetからConvNeXtへ
- **ConvNeXtの性能評価**
- ConvNeXt V2の概要
- FCMAE, GRN
- ConvNeXt V2の性能評価

ConvNeXtの性能評価

以下のように、チャンネル数とブロック数によりサイズの異なる5つのモデルを構築する

- ConvNeXt-T : C = (96,192,384,768)
B = (3,3,9,3)
- ConvNeXt-S : C = (96,192,384,768)
B = (3,3,27,3)
- ConvNeXt-B : C = (128,256,512,1024)
B = (3,3,27,3)
- ConvNeXt-L : C = (192,384,768,1536)
B = (3,3,27,3)
- ConvNeXt-XL : C = (256,512,1024,2048)
B = (3,3,27,3)



円の大きさはFLOPsを表す

Swin Transformerと比較して、各モデルサイズで精度が良くなっている

ConvNeXtの性能評価

● ConvNeXtとSwin Transformerの分類タスクでの性能比較

各サイズで、処理速度と精度が上がっている

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
● RegNetY-16G [54]	224 ²	84M	16.0G	334.7	82.9
● EffNet-B7 [71]	600 ²	66M	37.0G	55.1	84.3
● EffNetV2-L [72]	480 ²	120M	53.0G	83.7	85.7
○ DeiT-S [73]	224 ²	22M	4.6G	978.5	79.8
○ DeiT-B [73]	224 ²	87M	17.6G	302.1	81.8
○ Swin-T	224 ²	28M	4.5G	757.9	81.3
● ConvNeXt-T	224 ²	29M	4.5G	774.7	82.1
○ Swin-S	224 ²	50M	8.7G	436.7	83.0
● ConvNeXt-S	224 ²	50M	8.7G	447.1	83.1
○ Swin-B	224 ²	88M	15.4G	286.6	83.5
● ConvNeXt-B	224 ²	89M	15.4G	292.1	83.8
○ Swin-B	384 ²	88M	47.1G	85.1	84.5
● ConvNeXt-B	384 ²	89M	45.0G	95.7	85.1
● ConvNeXt-L	224 ²	198M	34.4G	146.8	84.3
● ConvNeXt-L	384 ²	198M	101.0G	50.4	85.5

ImageNet-22K pre-trained models					
● R-101x3 [39]	384 ²	388M	204.6G	-	84.4
● R-152x4 [39]	480 ²	937M	840.5G	-	85.4
● EffNetV2-L [72]	480 ²	120M	53.0G	83.7	86.8
● EffNetV2-XL [72]	480 ²	208M	94.0G	56.5	87.3
○ ViT-B/16 (📞) [67]	384 ²	87M	55.5G	93.1	85.4
○ ViT-L/16 (📞) [67]	384 ²	305M	191.1G	28.5	86.8
● ConvNeXt-T	224 ²	29M	4.5G	774.7	82.9
● ConvNeXt-T	384 ²	29M	13.1G	282.8	84.1
● ConvNeXt-S	224 ²	50M	8.7G	447.1	84.6
● ConvNeXt-S	384 ²	50M	25.5G	163.5	85.8
○ Swin-B	224 ²	88M	15.4G	286.6	85.2
● ConvNeXt-B	224 ²	89M	15.4G	292.1	85.8
○ Swin-B	384 ²	88M	47.0G	85.1	86.4
● ConvNeXt-B	384 ²	89M	45.1G	95.7	86.8
○ Swin-L	224 ²	197M	34.5G	145.0	86.3
● ConvNeXt-L	224 ²	198M	34.4G	146.8	86.6
○ Swin-L	384 ²	197M	103.9G	46.0	87.3
● ConvNeXt-L	384 ²	198M	101.0G	50.4	87.5
● ConvNeXt-XL	224 ²	350M	60.9G	89.3	87.0
● ConvNeXt-XL	384 ²	350M	179.0G	30.2	87.8

ConvNeXtの性能評価

● 物体検出・セグメンテーションでの性能

Mask R-CNN, Cascade Mask R-CNNを用いたCOCOでの物体検出(左)と、UperNetを用いたADE20Kでのセマンティックセグメンテーション(右)
いずれのサイズにおいてもSwin Transformerと同等以上の性能を発揮する

backbone	FLOPs	FPS	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅
Mask-RCNN 3× schedule								
○ Swin-T	267G	23.1	46.0	68.1	50.3	41.6	65.1	44.9
● ConvNeXt-T	262G	25.6	46.2	67.9	50.8	41.7	65.0	44.9
Cascade Mask-RCNN 3× schedule								
● ResNet-50	739G	16.2	46.3	64.3	50.5	40.1	61.7	43.4
● X101-32	819G	13.8	48.1	66.5	52.4	41.6	63.9	45.2
● X101-64	972G	12.6	48.3	66.4	52.3	41.7	64.0	45.1
○ Swin-T	745G	12.2	50.4	69.2	54.7	43.7	66.6	47.3
● ConvNeXt-T	741G	13.5	50.4	69.1	54.8	43.7	66.5	47.3
○ Swin-S	838G	11.4	51.9	70.7	56.3	45.0	68.2	48.8
● ConvNeXt-S	827G	12.0	51.9	70.8	56.5	45.0	68.4	49.1
○ Swin-B	982G	10.7	51.9	70.5	56.4	45.0	68.1	48.9
● ConvNeXt-B	964G	11.4	52.7	71.3	57.2	45.6	68.9	49.5
○ Swin-B [‡]	982G	10.7	53.0	71.8	57.5	45.8	69.4	49.7
● ConvNeXt-B [‡]	964G	11.5	54.0	73.1	58.8	46.9	70.6	51.3
○ Swin-L [‡]	1382G	9.2	53.9	72.4	58.8	46.7	70.1	50.8
● ConvNeXt-L [‡]	1354G	10.0	54.8	73.8	59.8	47.6	71.3	51.7
● ConvNeXt-XL [‡]	1898G	8.6	55.2	74.2	59.9	47.7	71.6	52.2

backbone	input crop.	mIoU	#param.	FLOPs
ImageNet-1K pre-trained				
○ Swin-T	512 ²	45.8	60M	945G
● ConvNeXt-T	512 ²	46.7	60M	939G
○ Swin-S	512 ²	49.5	81M	1038G
● ConvNeXt-S	512 ²	49.6	82M	1027G
○ Swin-B	512 ²	49.7	121M	1188G
● ConvNeXt-B	512 ²	49.9	122M	1170G
ImageNet-22K pre-trained				
○ Swin-B [‡]	640 ²	51.7	121M	1841G
● ConvNeXt-B [‡]	640 ²	53.1	122M	1828G
○ Swin-L [‡]	640 ²	53.5	234M	2468G
● ConvNeXt-L [‡]	640 ²	53.7	235M	2458G
● ConvNeXt-XL [‡]	640 ²	54.0	391M	3335G

- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- **ConvNeXt V2の概要**
- FCMAE, GRN
- ConvNeXt V2の性能評価

Vision Transformerの分野では、**MAE(Masked AutoEncoder)**という自己教師あり学習を事前学習として用いる手法が考案され、成功を収めた

しかし、このMAEをCNN系のモデルで事前学習として使用しようとしても上手くいかない



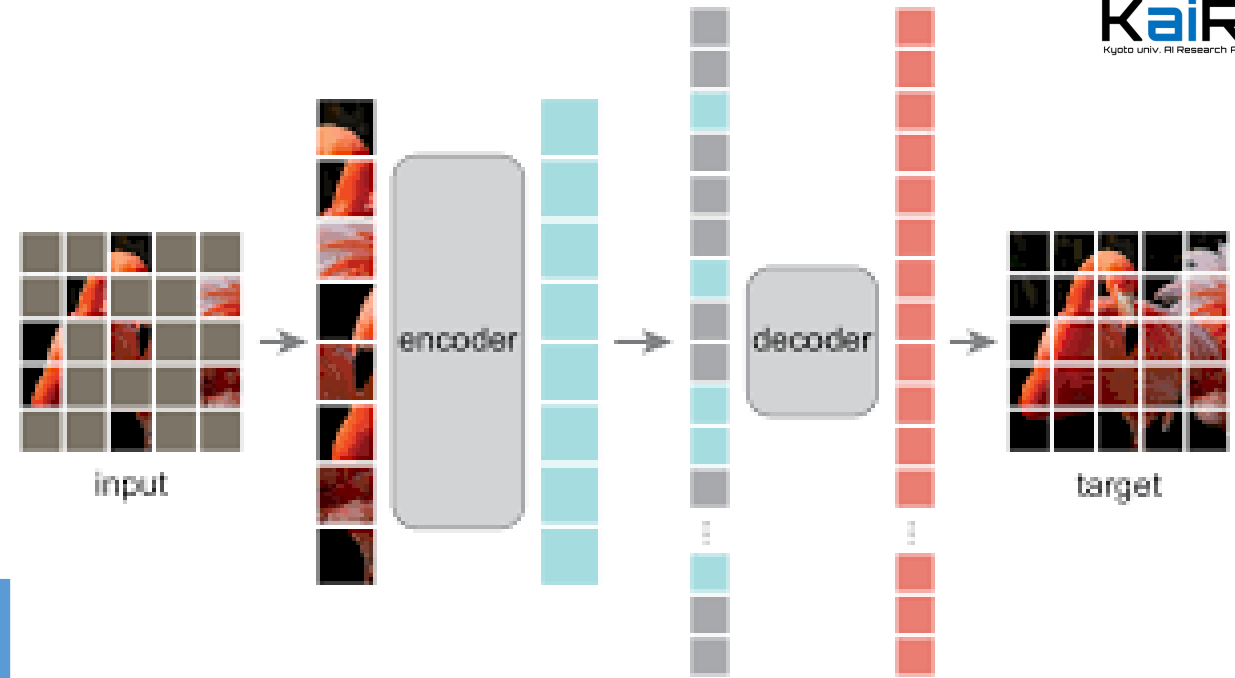
新たに、**FCMAE**という学習手法と、**GRN**という正規化層を導入し、MAEによる事前学習を取り入れたCNN系モデルを開発する

その結果、公開されたデータのみの学習で**ImageNetのSOTAを達成**

- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- ConvNeXt V2の概要
- **FCMAE, GRN**
- ConvNeXt V2の性能評価

● MAE(Masked AutoEncoder)

BERTの事前学習手法を元に提案された Vision Transformerの事前学習手法で、パッチの一部がマスクされた入力から元の状態を予測する



MAEがCNN系のモデルで上手くいかない原因

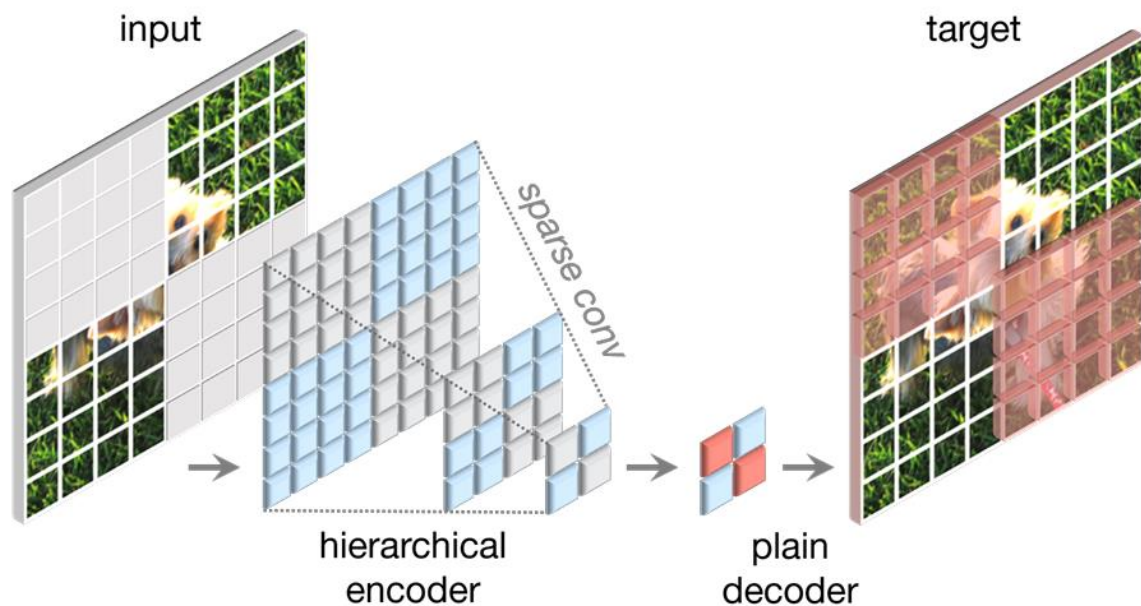
- マスクされた無駄な部分に多くの計算が必要となる
Transformerは、マスクされていない部分だけを入力として処理できるが、CNNではマスクした部分もそのまま入力しないといけない
- マスクされていない部分同士の間関係を学習しにくい
Transformerでは、Attentionによってマスクされていないパッチ間の間関係を直接学習できるが、CNNでは、マスクされた部分を多く取り込みながら関係性を学習する
→ マスクがあるという情報を学習するが、実用時はマスクは無い

FCMAE, GRU

■ FCMAE(Fully Convolutional Masked AutoEncoder)

1. ダウンサンプリングの過程でマスクされた領域が混ざらないように、 32×32 のパッチ化をしてから、60%をランダムにマスクする
2. エンコーダでは、**Sparse畳み込み**を使用し、
ファインチューニング時には普通の畳み込みに戻す
(点群データの知見を取り入れ、マスクされた画像を疎な行列と捉える)
3. デコーダはシンプルな構造を採用し、ConvNeXtの1ブロックのみを用いる

- マスク部分のリークがなくなる
- 計算量・メモリ使用量の削減

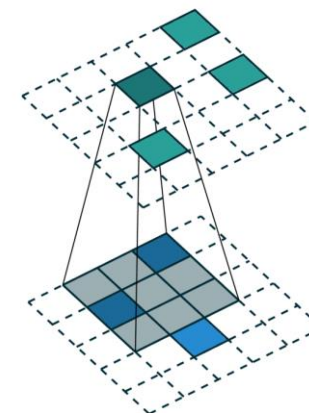
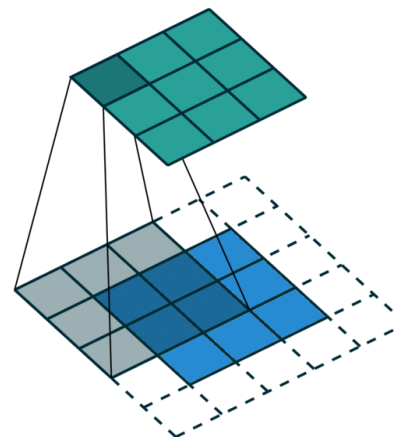


Sparse畳み込み

値のある部分のみを畳み込み演算する

通常の畳み込み

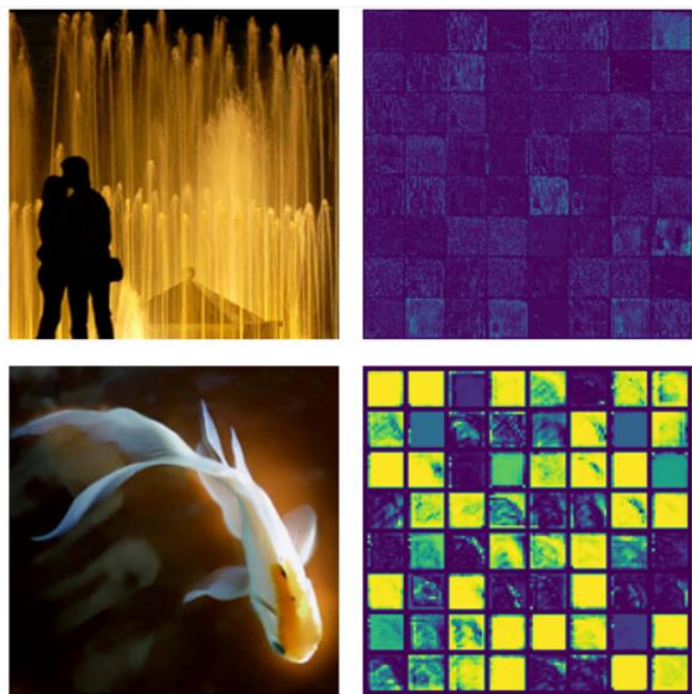
Sparse畳み込み



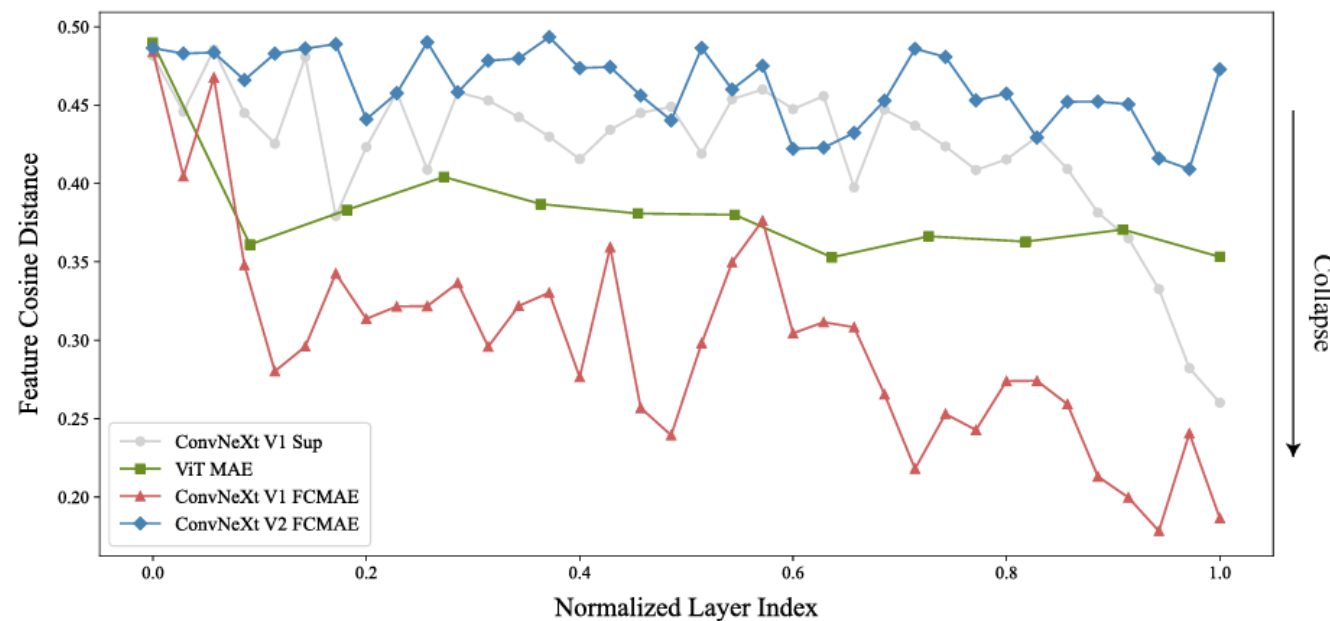
- 問題点：FCMAEで事前学習されたモデルでは、各チャンネルが似た特徴マップを出力してしまう**特徴崩壊**が起こっている



多様な特徴を学習できるように、特徴崩壊を防ぐ仕組みが必要になる



特徴マップの可視化



各層毎の特徴マップのコサイン距離

FCMAE, GRU

■ GRN(Global Response Normalization)

脳が持つ側方抑制(lateral inhibition)という、活性化されたニューロンが隣接するニューロンの活動を抑制する事により、ニューロンの多様性を促進する仕組みから提案された

1)大域的特徴集約 2)正規化 3)調整の3ステップで畳み込み層の出力をチャンネル間で正規化する

GRN

畳み込みの出力: $X \in \mathcal{R}^{H \times W \times C}$
i番目のチャンネル: $X_i \in \mathcal{R}^{H \times W}$
 $\|\cdot\|$: L2ノルム

$$1) \mathcal{G}(X) = (\|X_1\|, \|X_2\|, \dots, \|X_C\|) \in \mathcal{R}^C$$

$$2) \mathcal{N}(\|X_i\|) = \frac{\|X_i\|}{\sum_{j=1}^C \|X_j\|} \in \mathcal{R}$$

$$3) X_i = \gamma * X_i * \mathcal{N}(\mathcal{G}(X)_i) + \beta + X_i$$

($\gamma \cdot \beta$ は学習可能なパラメータ)

case	ft
g.avg.	83.7
L1	84.3
L2	84.6

(a) **Global aggregation** $\mathcal{G}(\cdot)$. L2 Norm-based aggregation function produces the best result.

case	ft
$(\ X_i\ - \mu)/\sigma$	84.5
$1/\sum \ X_i\ $	83.8
$\ X_i\ /\sum \ X_i\ $	84.6

(b) **Normalization operator**, $\mathcal{N}(\cdot)$. Divisive normalization is an effective channel importance calibrator.

case	ft
w/o skip	84.0
w/ skip	84.6

(c) **Residual connection** helps with GRN optimization and leads to better performance.

case	ft
Baseline	83.7
LRN [45]	83.2
BN [41]	80.5
LN [2]	83.8
GRN	84.6

(d) **Feature normalization**. GRN outperforms other normalizations through global contrasting.

case	ft	#param
Baseline	83.7	89M
SE [37]	84.4	109M
CBAM [72]	84.5	109M
GRN	84.6	89M

(e) **Feature re-weighting**. GRN does effective and efficient feature re-weighting without parameter overhead.

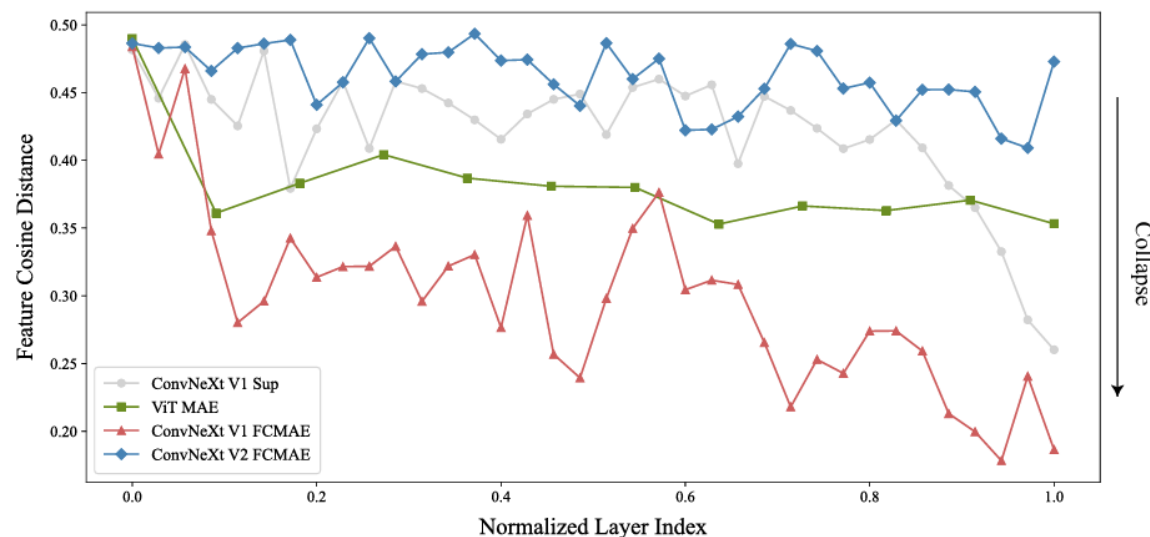
case	ft
Baseline	83.7
drop at ft.	78.8
add at ft.	80.6
both	84.6

(f) **GRN in pre-training/fine-tuning**. To be effective, GRN should be used in both stages.

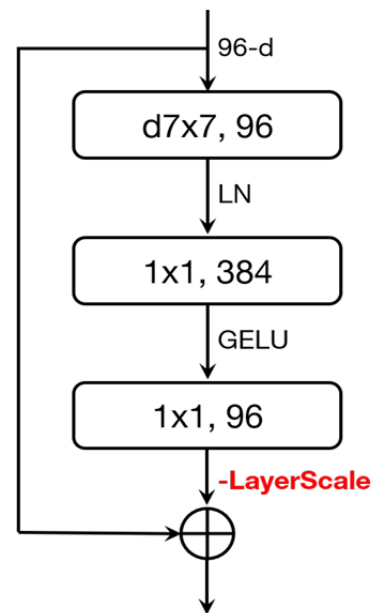
FCMAE, GRU

GRNは各ブロックの活性化関数GeLUの後に入れる
実験の結果、LayerScaleは外しても学習が進むようになり、
最終的に右のようなブロックになった

目的であった、畳み込みの出力の多様化が以下で確認できる

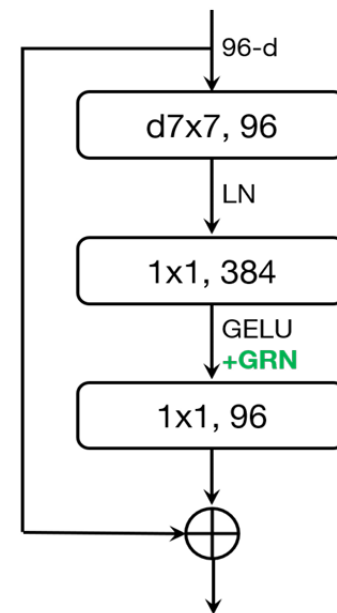


ConvNeXt V1 Block

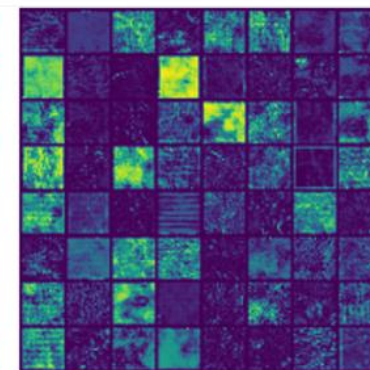
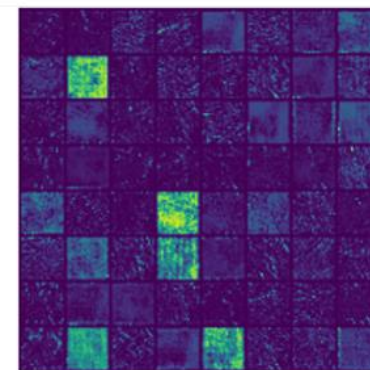
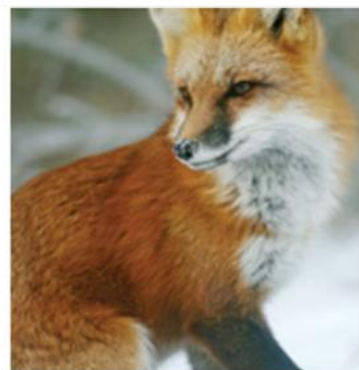


ConvNeXt V1

ConvNeXt V2 Block



ConvNeXt V2

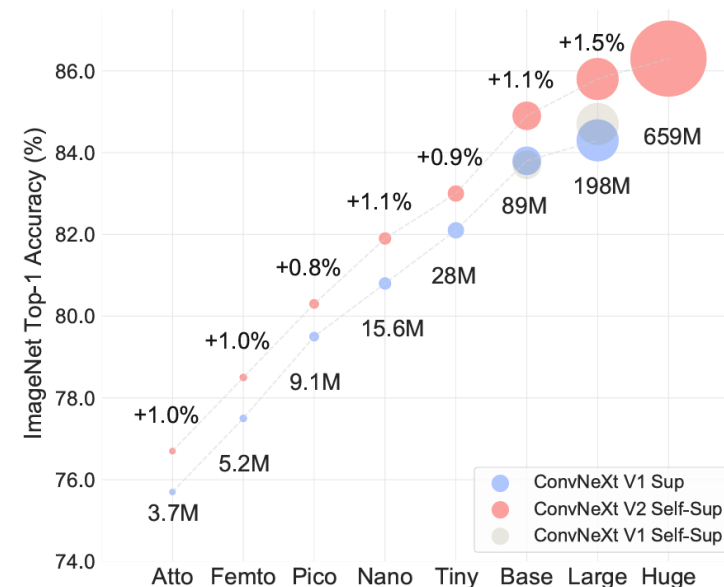


- ConvNeXtの概要
- ResNetからConvNeXtへ
- ConvNeXtの性能評価
- ConvNeXt V2の概要
- FCMAE, GRN
- ConvNeXt V2の性能評価

ConvNeXt V2の性能評価

以下の8種類の大きさの異なるモデルを提案

- ConvNeXt V2-A: $C=40, B=(2, 2, 6, 2)$
 - ConvNeXt V2-F: $C=48, B=(2, 2, 6, 2)$
 - ConvNeXt V2-P: $C=64, B=(2, 2, 6, 2)$
 - ConvNeXt V2-N: $C=80, B=(2, 2, 8, 2)$
 - ConvNeXt V2-T: $C=96, B=(3, 3, 9, 3)$
 - ConvNeXt V2-B: $C=128, B=(3, 3, 27, 3)$
 - ConvNeXt V2-L: $C=192, B=(3, 3, 27, 3)$
 - ConvNeXt V2-H: $C=352, B=(3, 3, 27, 3)$
- 全てのモデルサイズにおいて、ConvNeXt・Swin Transformerを上回った
 - ConvNeXtはスケーリング可能



Backbone	Method	#param	PT epoch	FT acc.
ViT-B	BEiT	88M	800	83.2
ViT-B	MAE	88M	1600	83.6
Swin-B	SimMIM	88M	800	84.0
ConvNeXt V2-B	FCMAE	89M	800	84.6
ConvNeXt V2-B	FCMAE	89M	1600	84.9
ViT-L	BEiT	307M	800	85.2
ViT-L	MAE	307M	1600	<u>85.9</u>
Swin-L	SimMIM	197M	800	85.4
ConvNeXt V2-L	FCMAE	198M	800	85.6
ConvNeXt V2-L	FCMAE	198M	1600	85.8
ViT-H	MAE	632M	1600	<u>86.9</u>
Swin V2-H	SimMIM	658M	800	85.7
ConvNeXt V2-H	FCMAE	659M	800	85.8
ConvNeXt V2-H	FCMAE	659M	1600	86.3

ConvNeXt V2の性能評価

■ ConvNeXt V2の転移学習性能

COCOでの、Mask R-CNNによる物体検出とセグメンテーション(左)

ADE20Kでの、UPerNetによるセマンティックセグメンテーション(右)

Backbone	Method	FLOPS	AP ^{box}	AP ₅₀ ^{box}	AP ₇₅ ^{box}	AP ^{mask}	AP ₅₀ ^{mask}	AP ₇₅ ^{mask}
ConvNeXt V1-B	Supervised	486G	50.3	71.6	56.1	44.9	68.5	48.8
ConvNeXt V2-B	Supervised	486G	51.0	72.4	56.6	45.6	69.5	49.7
Swin-B	SimMIM	497G	52.3	—	—	—	—	—
ConvNeXt V2-B	FCMAE	486G	52.9	72.6	58.9	46.6	70.0	51.1
ConvNeXt V1-L	Supervised	875G	50.6	71.5	56.3	45.1	68.7	49.2
ConvNeXt V2-L	Supervised	875G	51.5	72.5	57.3	45.8	69.4	49.9
Swin-L	SimMIM	904G	53.8	—	—	—	—	—
ConvNeXt V2-L	FCMAE	875G	54.4	73.9	60.4	47.7	71.4	52.3
Swin V2-H	SimMIM	—	54.4	—	—	—	—	—
ConvNeXt V2-H	FCMAE	2525G	55.7	75.2	61.8	48.9	72.8	53.6

Backbone	Method	input	mIoU	#param	FLOPS
ConvNeXt V1-B	Supervised	512 ²	49.9	122M	1170G
ConvNeXt V2-B	Supervised	512 ²	50.5	122M	1170G
Swin-B	SimMIM	512 ²	52.8	121M	1181G
ConvNeXt V2-B	FCMAE	512 ²	52.1	122M	1170G
ConvNeXt V1-L	Supervised	512 ²	50.5	235M	1573G
ConvNeXt V2-L	Supervised	512 ²	51.6	235M	1573G
Swin-L	SimMIM	512 ²	53.5	234M	1601G
ConvNeXt V2-L	FCMAE	512 ²	53.7	235M	1573G
Swin V2-H	SimMIM	512 ²	54.2	—	—
ConvNeXt V2-H	FCMAE	512 ²	55.0	707M	3272G
ConvNeXt V2-H	FCMAE, 22K ft	640 ²	57.0	707M	5113G

ConvNeXt

ResNetにSwin Transformerの手法を取り入れることで、
畳み込みのみでTransformer系を超える性能を達成

ConvNeXt V2

Sparse畳み込みと、GRNを導入したことにより、
マスクを復元する大規模な事前学習ができるようになった

京都大学人工知能研究会

KaiRA

Kyoto univ. AI Research Association